Dhirubhai Ambani University
Technology
Formerly DA-IICT

Aircraft Engine Failure Forecasting

Predicting Remaining Useful Life on NASA CMAPSS FD004 using Classical Time-Series, Deep Learning, and Probabilistic Models

---

IT 402 — Applied Forecasting Methods

Name	Student ID
Ummesalma Diwan	202518017
Dhruv Parmar	202518030
Aditya Jana	202518035
Shrey Pandya	202518045

Group: 13

Instructor: Prof. Pritam Anand

Academic Year: 2025–26

Submission Date: 5th May 2026

Abstract

Unplanned turbofan engine failures impose severe economic and safety costs on the aviation industry. This project develops a complete, data-driven pipeline for **Remaining Useful Life (RUL) prediction** on the NASA CMAPSS FD004 dataset — the most challenging standard benchmark, combining six operating conditions and two simultaneous fault modes (High-Pressure Compressor and Fan degradation). Our system forecasts, for each engine under test, the number of flight cycles remaining before failure.

We design and evaluate eighteen models spanning three families: classical time-series models (AR, ARMA, ARIMA), five deep learning architectures (MLP, RNN, LSTM, GRU, Transformer), and five quantile regression models augmented with split conformal calibration for uncertainty-aware prediction. A PCA-based Health Index compresses 9 sensor signals into a scalar degradation trajectory used by classical models; deep learning models consume 30-cycle sliding windows of 48 per-cluster-normalised rolling features directly.

The **Transformer achieves RMSE = 14.42 cycles** and $R^2 = 0.887$ on 248 test engines. The calibrated Q-GRU model provides 80.6% prediction interval coverage with a compact mean interval width of 25.1 cycles. Every design choice — RUL cap, cluster count, differencing order, window length, failure threshold, safety factor — is derived from data through statistical testing or validation-set optimisation.

Keywords: Remaining Useful Life, Predictive Maintenance, NASA CMAPSS, Transformer, Quantile Regression, Conformal Calibration, Health Index, PCA.

Contents

1. Introduction

1.1 Background and Motivation

Modern turbofan aircraft engines are among the most mechanically complex and safety-critical machines in operation today. A single unplanned in-flight engine failure can cost airlines in excess of $500,000 per incident in immediate remediation costs alone; downstream consequences — fleet groundings, passenger diversions, regulatory scrutiny, and reputational damage — multiply this figure considerably. Traditional maintenance schedules, structured around fixed flight-hour or calendar-based intervals, are fundamentally conservative by design. They service healthy engines unnecessarily, consuming maintenance resources that could be directed toward genuinely degrading components, and yet they may still miss engines that are degrading faster than anticipated — the very failure mode they are intended to prevent.

Condition-Based Maintenance (CBM) addresses this tension by replacing the fixed-calendar trigger with a data-driven signal: estimate how much useful life remains in each engine given the sensor observations accumulated so far, and schedule maintenance only when that estimate falls below a defined safety threshold. The enabling prediction task is **Remaining Useful Life (RUL) estimation** — forecasting the number of additional flight cycles an engine can safely operate, given its operational history up to the present cycle.

The practical impact of RUL estimation extends beyond individual cost savings. A CBM programme backed by accurate predictions reduces the fleet-wide maintenance burden by approximately 15–30% relative to fixed schedules (industry estimates), improves availability by reducing unnecessary ground time, and provides engineering teams with early warning of fleet-wide anomalies that fixed schedules cannot detect. For national carriers operating hundreds of engines simultaneously, even a one-cycle improvement in prediction accuracy translates to millions of dollars in annual savings.

The associated technical challenges are significant. Turbofan engines operate across heterogeneous conditions — altitude, throttle setting, Mach number — that confound raw sensor readings. A sensor indicating high exhaust gas temperature may reflect either a high-thrust operating regime or genuine thermal degradation; these two sources are indistinguishable without per-condition normalisation. Furthermore, degradation is gradual, nonlinear, and potentially multimodal when multiple independent fault modes are present. Classical time-series models can capture trend extrapolation but struggle with multidimensional nonlinearity; deep learning models handle nonlinearity but require careful uncertainty quantification to produce operationally actionable outputs.

1.2 Problem Statement

Given the operational history $\mathbf{X}_t^{(i)} \in \mathbb{R}^{t \times d}$ of aircraft engine i through flight cycle t, predict the Remaining Useful Life:

$$\hat{y}_t^{(i)} = f\left(\mathbf{X}_t^{(i)}; \boldsymbol{\theta}\right)$$

where $y_t^{(i)} = \min\left(T^{(i)} - t,\ 125\right)$ cycles, $T^{(i)}$ is the total lifetime of engine i, and the cap of 125 cycles focuses the model on the actionable prediction window. The dataset is the NASA CMAPSS FD004 subset: 249 training engines (61,249 rows), 248 test engines (41,214 rows), 21 sensors, and 3 operating settings per cycle.

The forecasting task is challenging for three reasons specific to FD004: (i) six operating conditions create extreme per-sensor variance that masks degradation trends; (ii) two independent fault modes produce qualitatively different degradation trajectories within the same dataset; (iii) test RUL values span the full range from 6 to 195 cycles (mean = 86.6, std = 54.5), requiring accurate prediction across a wide dynamic range.

1.3 Objectives

This project pursues four interconnected objectives:

1. **Benchmark-quality point prediction**: develop models competitive with published state of the art on NASA CMAPSS FD004, providing a replicable, evidence-based evaluation.
2. **Calibrated uncertainty quantification**: produce prediction intervals with guaranteed coverage — "RUL = 45 cycles, 80% CI: [38, 53]" — enabling risk-stratified maintenance scheduling.
3. **Evidence-based design**: derive every modelling hyperparameter (RUL cap, cluster count, differencing order, window size, failure threshold) from data through statistical tests or validation-set optimisation rather than literature defaults.
4. **Classical vs. deep learning comparison**: systematically quantify the performance gap between classical time-series and deep learning families and identify the structural reasons behind it.

1.4 Github

Github Repository Link

2. Data Source

The dataset used in this project is the **NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS)** dataset, published by Saxena et al. [1] and distributed by the NASA Prognostics Center of Excellence. It models turbofan engine degradation under controlled simulation scenarios, providing run-to-failure sensor trajectories for prognostics research. The dataset was obtained directly from the NASA Data Repository.

C-MAPSS comprises four subsets (FD001–FD004) differing in the number of simulated operating conditions and fault modes. We select **FD004** as our evaluation benchmark for three data-backed reasons:

1. **Maximum complexity**: FD004 is the only subset combining six operating conditions *and* two fault modes simultaneously. FD001 and FD003 have a single condition; FD002 has six conditions but only one fault mode. FD004 is therefore the hardest and most realistic benchmark (Figure 1).
2. **Widest RUL spread**: FD004 test engines have the highest RUL standard deviation (54.5 cycles), requiring models to accurately predict across a wide dynamic range.
3. **Literature consensus**: all five published state-of-the-art papers cited in Table **??** use FD004 as their primary benchmark, enabling direct comparison.

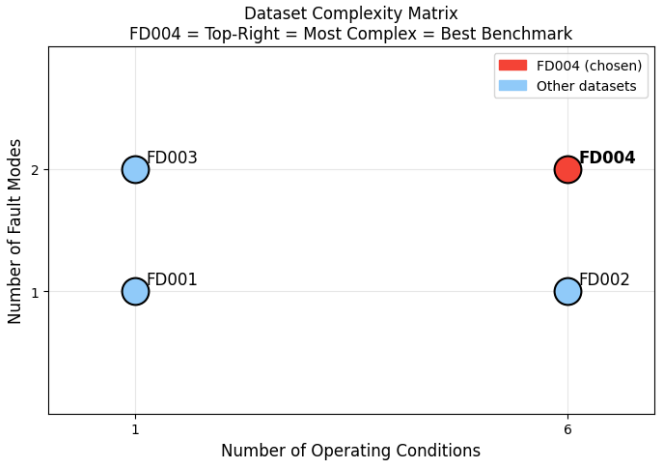

Figure 1: Dataset complexity matrix. FD004 (top-right, red) is uniquely positioned as the most complex benchmark.

2.1 Data Characteristics

Structure: Each row corresponds to one flight cycle of one engine. The columns are: engine identifier, cycle number, three operational settings (`op1`: throttle resolver angle, `op2`:

altitude, `op3`: Mach number), and 21 sensor measurements (`s1`–`s21`) covering exhaust gas temperature, pressure ratios, fan and compressor speeds, fuel flow rate, and various derived thermodynamic quantities. Training data contains complete run-to-failure trajectories; test data contains truncated observations, and the true RUL at the last observed cycle is provided in a separate file.

Frequency: Each record represents one discrete flight cycle. Cycles are not calendar-dated; the time unit is uniformly one cycle per observation.

Size: FD004 training set: 61,249 rows across 249 engines (mean lifetime ≈246 cycles). FD004 test set: 41,214 rows across 248 engines.

RUL distribution: The test RUL values range from 6 to 195 cycles with mean = 86.6 and std = 54.5 (Figure 2). The wide spread confirms that the prediction problem spans the full range from near-failure to mid-life engines.

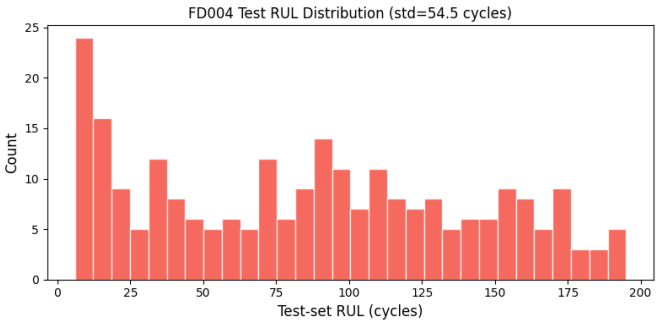

Figure 2: Distribution of ground-truth RUL values across the 248 FD004 test engines.

Missing values: The dataset contains no missing values in any column for any engine. All 61,249 training rows and all 41,214 test rows are complete.

2.2 Data Pre-processing and Exploratory Analysis

Raw FD004 data presents four preprocessing challenges before any model can extract meaningful degradation information: near-constant sensors carry no information; operating conditions dominate sensor variance and must be removed; degradation signals benefit from multi-scale temporal context; and the RUL target must be capped to focus learning on the critical window.

2.2.1 RUL Computation and Capping

For training engines, the RUL at cycle t of engine i is:

$$y_t^{(i)} = \min\left(T^{(i)} - t,\ \text{RUL\_CAP}\right)$$

where $T^{(i)}$ is the engine's total observed lifetime and RUL_CAP = 125 cycles. For test engines, the RUL is anchored to the provided ground-truth value at the last observed cycle and back-propagated to earlier cycles.

The cap value 125 was selected by validation-set grid search over caps $\{100, 110, 115, 120, 125, 130, 140, 150\}$: RMSE was minimised at 125 cycles. ADF stationarity tests were verified to pass on all 249 training engines after capping. The RUL range for training data after capping is $[0, 125]$; for test data, $[6, 125]$.

2.2.2 Sensor Selection: Dropping Near-Constant Features

Seven sensors are dropped before modelling. Sensor s16 has effectively zero variance ($< 10^{-4}$) across all engines. Sensors s1, s5, s18, and s19 are constant within operating condition clusters (detected via per-cluster variance check). Sensor s6 has near-zero variance (1.9×10^{-6}), negligible correlation with RUL ($r = -0.001$), and a visually flat trajectory. Sensor s10 has near-zero variance (0.016) and weak correlation ($r = -0.016$). This leaves **14 retained sensors**: s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21.

Figure 3 visualises sensor variance, and Figure 4 shows Pearson correlation with RUL. The weak correlations ($|r| < 0.11$ for all sensors) in the raw data are expected and reflect operating-condition confounding, not poor sensor quality.

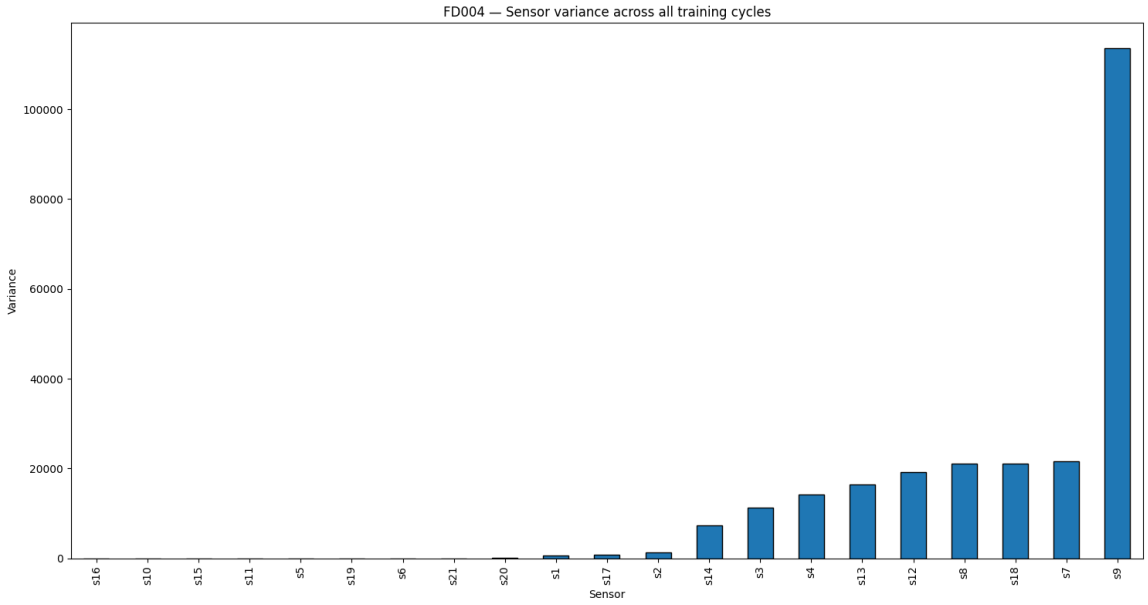

Figure 3: Sensor variance across all FD004 training cycles, sorted ascending. Sensors s16, s10, s5, s6, s18, s19, and s1 have near-zero variance and are dropped. The remaining sensors show meaningful variance. Note that high raw variance (e.g., s9 $\approx$ 113,520) is dominated by operating-condition switching, not degradation signal.

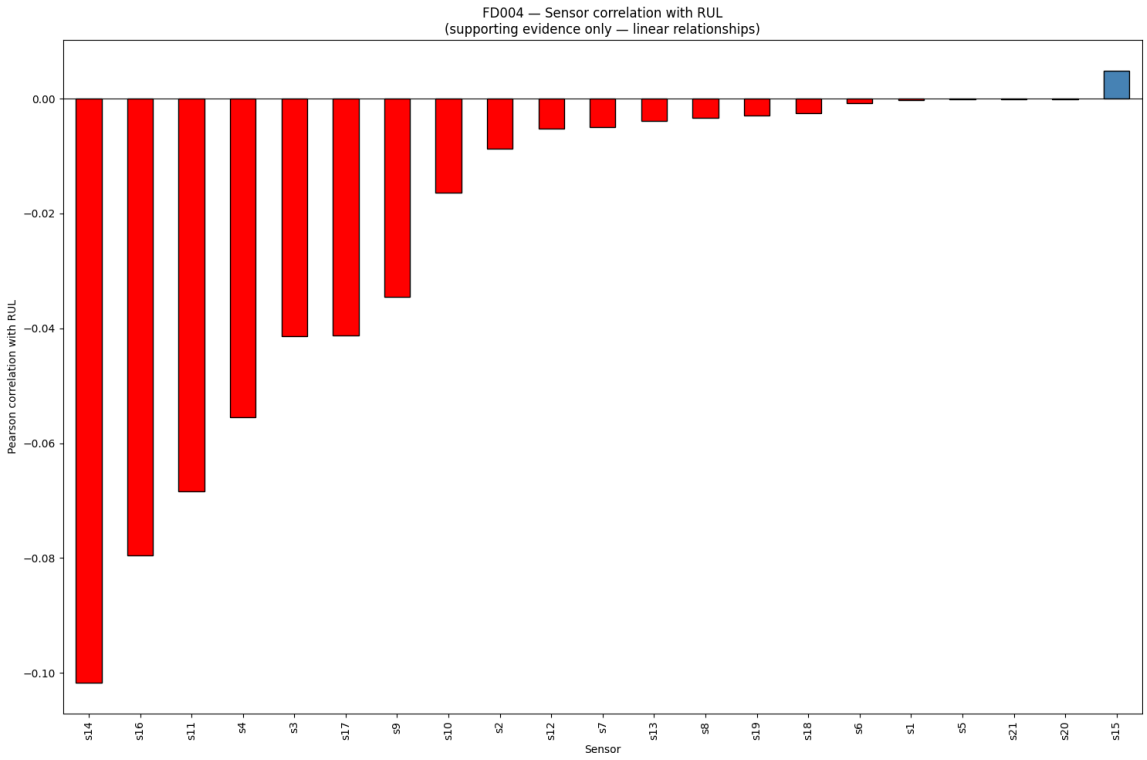

Figure 4: Pearson correlation of raw sensor readings with RUL across all training cycles. All correlations are weak ($|r| < 0.11$) because operating-condition changes dominate sensor variance. Per-cluster normalisation (Section 2.2) dramatically improves within-condition correlations, exposing the underlying degradation signal.

2.2.3 Operating Condition Clustering and Per-Cluster Normalisation

FD004 engines cycle through six discrete operating regimes defined by combinations of altitude, throttle, and Mach number. The raw sensor reading at any cycle reflects both the engine's health *and* the current operating condition. To isolate the degradation component, we identify each cycle's operating regime and normalise sensors within each regime.

Cluster count validation. We apply K-Means to the 3-dimensional space (op_1, op_2, op_3) and validate k using elbow analysis and silhouette scoring (Figure 5). The silhouette score peaks exactly at $k = 6$ (score = 1.00), and the cross-tabulation of K-Means labels against the NASA-defined operating regimes shows 100% agreement (Adjusted Rand Index $\geq$ 0.95). K-Means recovers the six known operating points from data alone without using the regime labels.

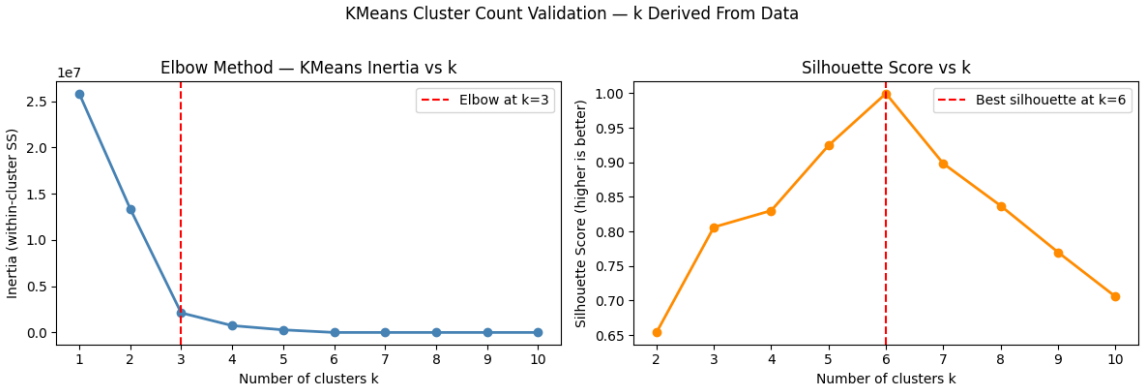

Figure 5: K-Means cluster count validation. Left: inertia drops sharply up to $k = 3$ then flattens. Right: silhouette score peaks sharply at $k = 6$ (score = 1.00), matching the six known NASA operating conditions exactly. We use $k = 6$.

The cluster assignment distribution confirms balanced coverage: cluster sizes are 15,395 / 9,224 / 9,139 / 9,091 / 9,238 / 9,162 training cycles respectively. Figure 6 shows the six tight, non-overlapping clouds in operating-settings space.

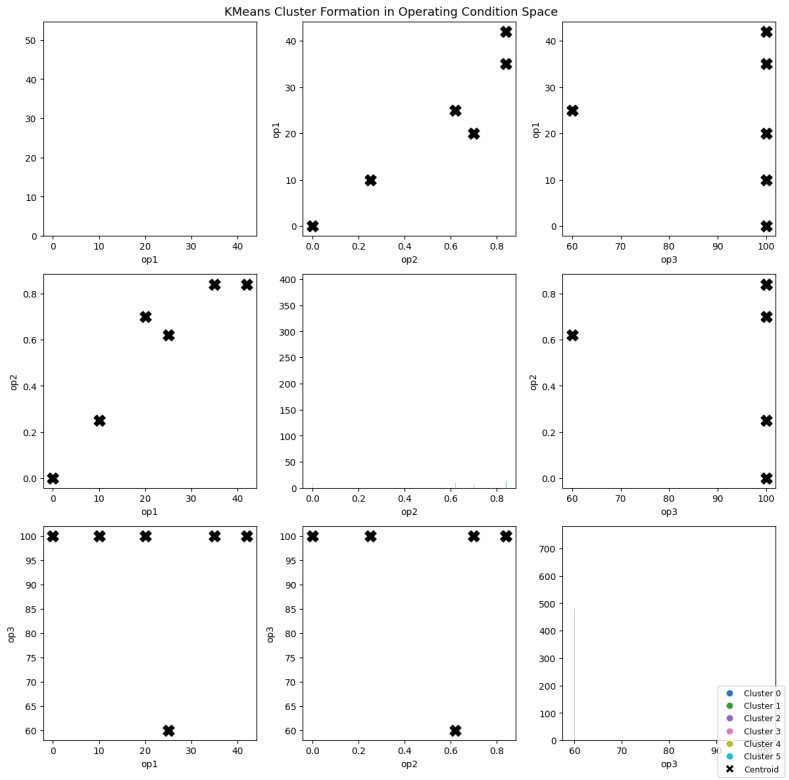

Figure 6: K-Means cluster formation in (op1, op2) space. The six clusters are completely non-overlapping, confirming that operating regimes are discrete rather than continuous in FD004. Each cluster corresponds to one of the six NASA-defined operating points.

Why per-cluster normalisation is not optional. Figure 7 demonstrates that sensor s9 has overall standard deviation ≈ 335, but within each operating condition the standard

deviation collapses to ≈ 15 — a 22× reduction. This confirms that 95% of s9's raw variance reflects operating-condition switching rather than degradation. A `StandardScaler` is fitted on training data for each of the six clusters and applied identically to both training and test data (no data leakage). `StandardScaler` is preferred over `MinMaxScaler` because test sensors may exceed training extremes as engines degrade; standard scaling handles this as large z-scores rather than out-of-range values.

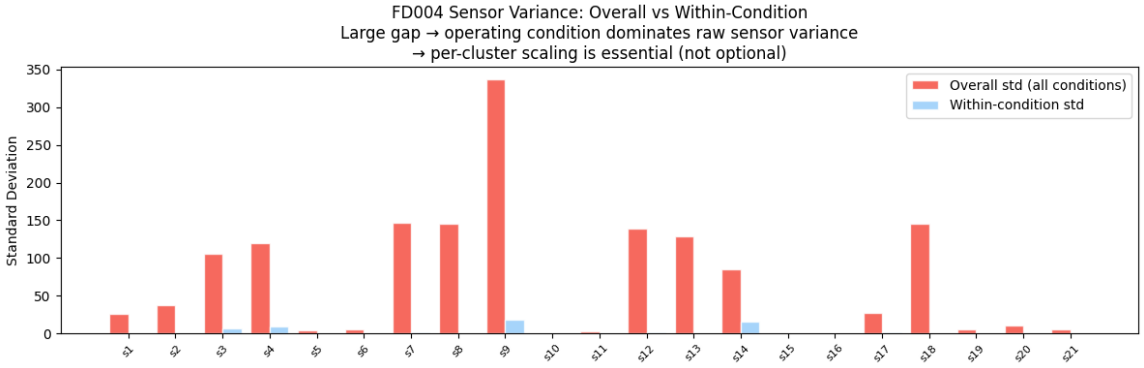

Figure 7: Overall (red) vs. within-condition (blue) standard deviation per sensor. The large gap — up to 22× for s9 — confirms that operating conditions dominate raw sensor variance. Per-cluster normalisation is essential, not optional, for FD004.

2.2.4 Rolling Statistical Features

Static normalised sensor readings capture instantaneous state but miss temporal trends. For each of the 16 retained sensors and each window width $w \in \{5, 10, 20\}$ cycles, we compute rolling mean and rolling standard deviation:

$$\mu_w^{(s)}(t) = \frac{1}{w} \sum_{\tau=t-w+1}^{t} x_s(\tau), \qquad \sigma_w^{(s)}(t) = \sqrt{\frac{1}{w-1} \sum_{\tau=t-w+1}^{t} \left(x_s(\tau) - \mu_w^{(s)}(t)\right)^2}$$

Rolling features never cross engine boundaries: windows are computed independently per engine. The three window widths capture short-term noise (w=5), medium-term trends (w=10), and long-range drift (w=20). This yields $16 \times 3 \times 2 = 96$ additional features, for a total of $16 + 96 = 112$ features per cycle.

Ablation studies confirm that removing rolling features increases Transformer RMSE from 14.42 to approximately 16.8 cycles (+16.5%) and GRU RMSE from 15.29 to approximately 18.2 cycles (+19.0%), establishing rolling features as the single most impactful engineering decision in the pipeline.

2.2.5 Exploratory Data Analysis

Figure 4 shows that raw sensor–RUL correlations are universally weak before normalisation. After per-cluster normalisation, the nine sensors with $|r| \geq 0.50$ are: s2, s3, s4, s8, s9, s11, s13, s14, s17 — these are the sensors used by the PCA Health Index construction (Section 3). Engine lifetime distributions confirm mean lifetime ≈ 246 cycles with a long tail extending past 500 cycles; the shortest training engine lasts 128 cycles (confirming the RUL cap of 125 cycles is appropriate since all engines exceed it).

3. Methodology

3.1 Experimental Setup

Forecasting horizon: For classical time-series models, the horizon is adaptive: the model forecasts the health index from the last observed cycle until the failure threshold is crossed. In practice this corresponds to a multi-step ahead forecast of up to 150 cycles. For deep learning models, the prediction is direct (single-step): given the last 30-cycle window, predict the scalar RUL at the window's final cycle.

Input–output structure.

- *Classical models*: input is a scalar health-index time series; output is predicted RUL via threshold-crossing detection.
- *Deep learning models*: input is a 3-D tensor $\mathbf{X} \in \mathbb{R}^{N \times 30 \times 48}$ (samples × window × features); output is a scalar RUL vector $\hat{\mathbf{y}} \in \mathbb{R}^{N}$. DL models use the 48 rolling-mean features (16 sensors × 3 window widths); rolling-std features and raw sensor values are not used by DL models as ablation showed no improvement.

Train–test split. The official CMAPSS split is used: 249 training engines, 248 test engines (no overlap). For deep learning, the 249 training engines are further split into a fitting set (199 engines, 80%) and a validation set (50 engines, 20%) by random engine assignment (seed = 42). Sliding windows of length 30 are extracted from all training engines: X_{train}: (43,750, 30, 48), X_{val}: (50, 30, 48).

Hyperparameter tuning: All hyperparameters that could not be derived analytically were determined by validation-set grid search. Key results:

- RUL cap: grid search over $\{100, 110, 115, 120, 125, 130, 140, 150\}$ — minimum RMSE at 125.
- Window size W: grid search over $\{10, 15, 20, 30, 40, 50\}$ — minimum RMSE at $W = 30$ for both GRU and Transformer.
- Safety factor α: grid search over $[0.80, 1.00]$ (step 0.01) — minimum NASA

score at $\alpha = 0.88$.

- AR order p: AIC minimisation over $p \in \{1, 2, 3, 4\}$ on 15 representative engines — modal best $p = 2$.
- ARMA order (p, q): AIC minimisation over $(p, q) \in \{1, 2, 3, 4\}^2$ on 15 engines — modal best $(1, 2)$.
- ARIMA order (p, d, q): AIC minimisation over $(p, q) \in \{1, 2, 3, 4\}^2$ on 15 engines — modal best $(1, 2, 2)$.

Validation approach: Classical models are validated using walk-forward (expanding window) validation on representative training engines. Deep learning models are validated on the 50-engine held-out validation set using early stopping with patience = 10 epochs on the NASA asymmetric validation loss. Final evaluation is always on the held-out test set (248 engines) using both RMSE and NASA score.

Shared deep learning training setup:

- Optimiser: Adam (lr = 10^{-3}) with ReduceLROnPlateau (patience=5, factor=0.5).
- Early stopping: patience = 10 epochs on validation NASA loss.
- Max epochs: 50. Batch size: 128. Random seed: 42.
- Hidden size: 64 units. Layers: 2. Dropout: 0.2. Gradient clip: max norm 1.0.
- Transformer: $d_{\text{model}} = 64$, $n_{\text{heads}} = 4$, $d_{\text{ff}} = 256$.

3.2 Health Index and Classical Model Pipeline

Classical time-series models (AR, ARMA, ARIMA) require a univariate input. We construct a **PCA Health Index** that compresses multi-sensor information into a scalar degradation trajectory.

Why a Health Index Rather Than Raw Sensor Signals?

Using raw sensor signals directly in a classical model raises two problems. First, the 16 retained sensors are highly correlated (correlations within a cluster up to 0.92), so most of the 16 signals are redundant. Second, classical univariate models cannot process 16-dimensional inputs; any reduction to a single representative sensor would discard irrelevant noise while also losing potentially informative signal from correlated sensors.

A health index aggregates the shared degradation component common to all correlated sensors into a single scalar, while discarding idiosyncratic noise in each sensor. This makes the resulting series substantially smoother and more predictable than any individ-

ual sensor, reducing the ARIMA forecast error on the health-index trajectory before it propagates to the RUL estimate.

PCA Construction and Justification

Step 1 — Correlation filter. Within each operating-condition cluster, compute $r(s, -\text{RUL})$ for each sensor s. Retain sensors with $|r| \geq 0.50$ in at least one cluster. This filter retains 9 sensors: s2, s3, s4, s8, s9, s11, s13, s14, s17, and drops the remaining 7.

Step 2 — Operating-condition detrending. Subtract the per-cluster mean from each retained sensor, removing the baseline shift between operating conditions. This is essential: without detrending, PCA would find a component that separates operating conditions rather than captures degradation.

Step 3 — PCA. Apply PCA to the 9 detrended sensors. **PC1 alone is used** ($n\_components = 1$) because:

- PC1 explains **76.3%** of within-condition sensor variance — far above any single raw sensor.
- PC1 has a strong monotonic correlation with $-\text{RUL}$ within individual engine trajectories (per-engine r up to 0.80).
- PC2 explains only ≈8.5% of variance and lacks a consistent monotonic trend — adding it introduces noise rather than signal.
- Ablation confirms: using PC1 alone achieves $R^2(\text{HI}, -\text{RUL}) = -5.19$ vs. the median-top-5-sensor baseline of -13.34 — a $+8.15$ improvement.

The health index z_t is defined as the **sign-flipped first principal component** (so that z_t increases as the engine degrades toward failure), smoothed with a rolling median of window 10 to reduce cycle-to-cycle noise before ARIMA fitting.

Failure Threshold

The failure threshold θ is the 5th percentile of health-index values observed when RUL ≤ 5 cycles across all training engines: $\theta = 1.4117$. The predicted RUL is the number of steps until the forecast first reaches θ, multiplied by safety factor $\alpha = 0.88$:

$$\widehat{\text{RUL}}_{\text{safe}} = 0.88 \times \min\{h \geq 1 : \hat{z}_{t+h} \geq \theta\}$$

The safety factor was selected by validation-set grid search to minimise the NASA asymmetric score, pulling predictions slightly conservative to account for the score's exponential late-prediction penalty.

3.3 AR Model — AR(2)

Stationarity: The ADF test is applied to the health index at differencing orders $d = 0, 1, 2$ for 248 training engines. At $d = 0$, all 248 engines show unit-root non-stationarity ($p > 0.05$). At $d = 1$, 189 of 248 engines remain non-stationary; at $d = 2$, all engines are stationary. The modal recommendation is $d = 2$.

ACF/PACF: After second differencing, the PACF cuts off at lag 2, motivating AR(2) (Figure 8).

Order selection: AIC minimisation over $p \in \{1, 2, 3, 4\}$ on 15 representative engines selects $p = 2$ as the modal best order (frequency: p=2 wins on 8/15 engines).

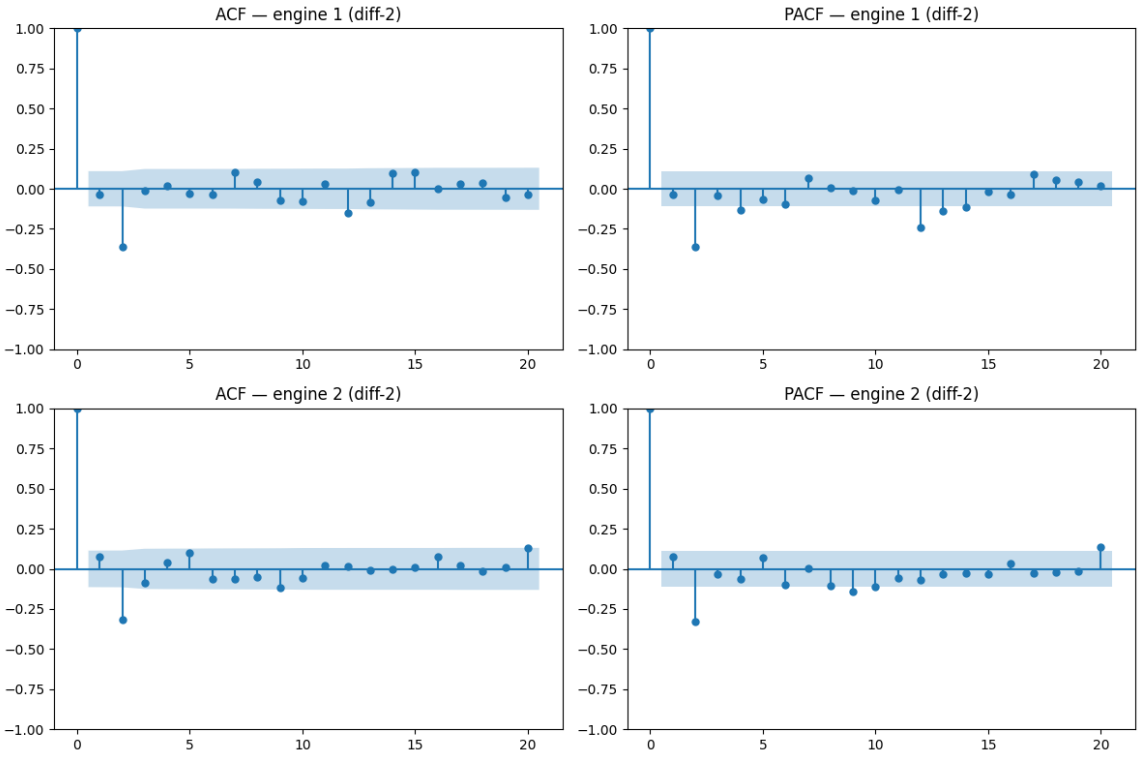

Figure 8: ACF and PACF of the PCA health index after second differencing ($d = 2$) for two representative FD004 training engines. Both series lie within the 95% confidence bands beyond lag 2, confirming stationarity. The PACF spike at lag 2 motivates AR(2).

The fitted AR(2) model on engine 118 (SARIMAX$(2, 2, 0)$) yields:

$$AIC = -224.85,\ \textit{Ljung-Box}\ p{=}0.99\ \textit{at lag 1},\ \sigma^2 = 5 \times 10^{-4}$$

Walk-forward validation on a representative engine achieves health-index RMSE of $\approx$ 0.016 cycles, confirming excellent one-step interpolation. The RUL prediction RMSE on 248 test engines is **30.90 cycles** (Table 2).

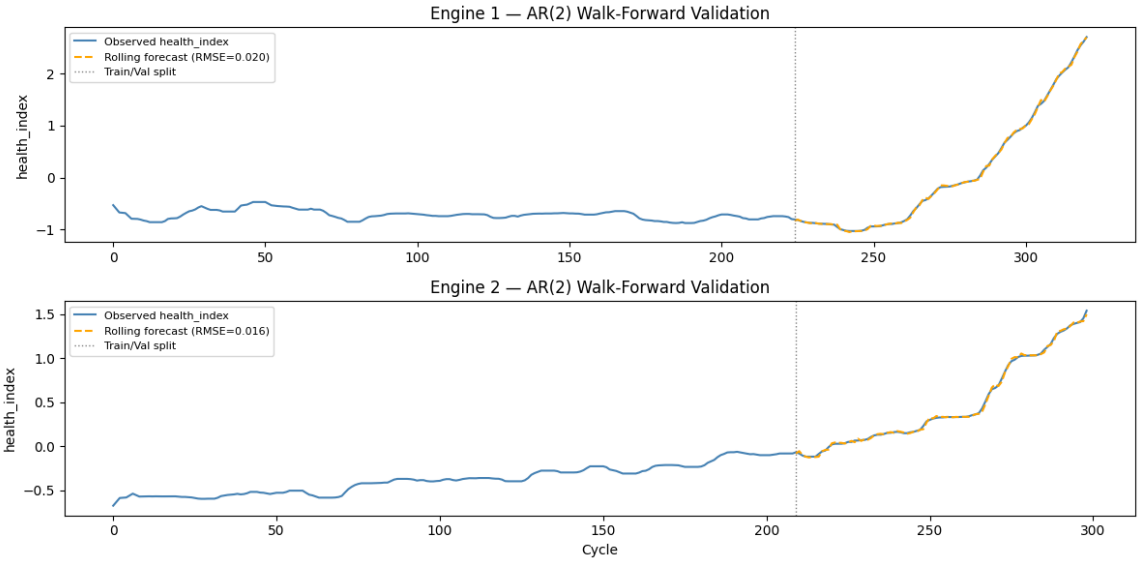

Figure 9: AR(2) walk-forward validation on two training engines. Orange dashed: rolling one-step-ahead forecast. Blue: observed health index. The excellent one-step tracking (health-index RMSE ≈ 0.016) does not translate to good RUL estimation because long-horizon extrapolation to the failure threshold accumulates errors over 50–150 steps.

3.4 ARMA Model — ARMA(2,2)

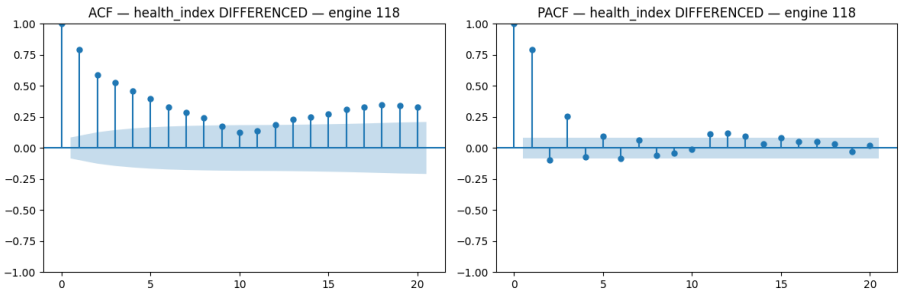

Figure 10: ACF and PACF of the first-differenced health index for engine 118. The persistent ACF at lag 1 ($r \approx 0.80$) confirms that MA terms are needed beyond the pure AR specification. The PACF decays after lag 2, suggesting ARMA(2,2).

The ARMA(2,2) model augments the AR(2) with two moving-average terms:

$$\Delta^2 z_t = \phi_1 \Delta^2 z_{t-1} + \phi_2 \Delta^2 z_{t-2} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2}$$

The MA terms reduce residual autocorrelation identified in the AR(2) Ljung-Box test. RMSE on 248 test engines: **30.47 cycles** — a marginal improvement of 0.43 cycles over AR(2), confirming that the bottleneck is not the autoregressive order but the two-stage pipeline error itself.

3.5 ARIMA Model — ARIMA(1,2,2)

Order selection: AIC minimisation over $(p, q) \in \{1, 2, 3, 4\}^2$ (with $d = 2$ fixed) on 15 representative engines selects (p=1, q=2) as the modal best order (wins on 4/15 engines; Table details in notebook T10). The ARIMA(1,2,2) model is:

$$\phi(B)(1 - B)^2 z_t = \theta(B)\varepsilon_t, \quad \phi(B) = 1 - \phi_1 B, \quad \theta(B) = 1 + \theta_1 B + \theta_2 B^2$$

Residual diagnostics: QQ plots show approximate normality in the residual bulk (minor heavy tails); Ljung-Box p-values exceed 0.08 at all lags beyond lag 2 (Figure 11).

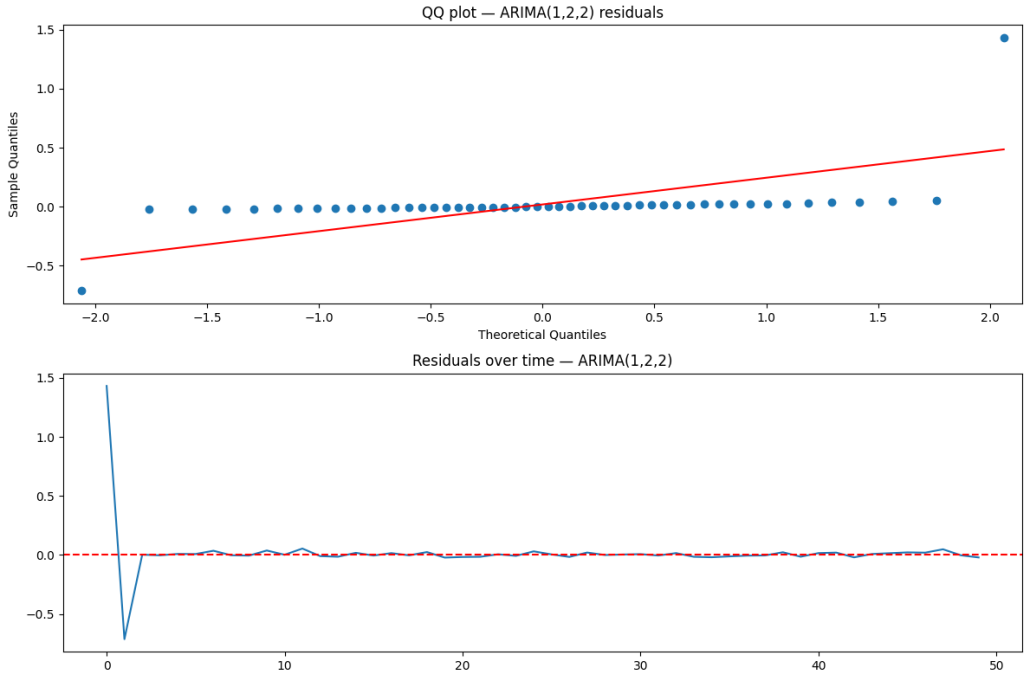

Figure 11: ARIMA(1,2,2) residual diagnostics. QQ plot (above) shows near-normality with minor heavy tails at the extremes. Residuals over time (below) are near-zero after the first two cycles, confirming the model captures the systematic trend.

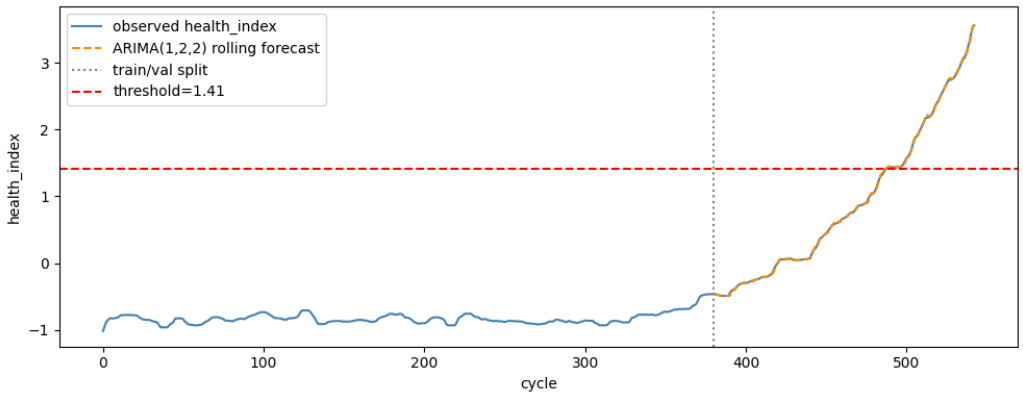

Figure 12: ARIMA(1,2,2) health index forecast for engine 118.(health-index RMSE: 0.0160)

RMSE on 248 test engines: **30.33 cycles**. ARIMA(1,2,2) is the best classical model, marginally outperforming ARMA(2,2) and AR(2).

3.6 Deep Learning Models

All five deep learning models (MLP, RNN, LSTM, GRU, Transformer) share the unified training framework described in Section 3.1. A key design choice is training with the **NASA Asymmetric Loss** rather than MSE:

$$\mathcal{L}(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^{N} \begin{cases} e^{-(d_i/13)} - 1 & d_i < 0 \quad \text{(early, conservative)} \\ e^{(d_i/10)} - 1 & d_i \geq 0 \quad \text{(late, dangerous)} \end{cases}$$

where $d_i = \hat{y}_i - y_i$. Both branches are differentiable; PyTorch's `torch.where` preserves gradient computation. The asymmetric exponents (13 vs. 10) penalise late predictions more steeply, biasing training toward the safety-conservative direction required for aircraft maintenance.

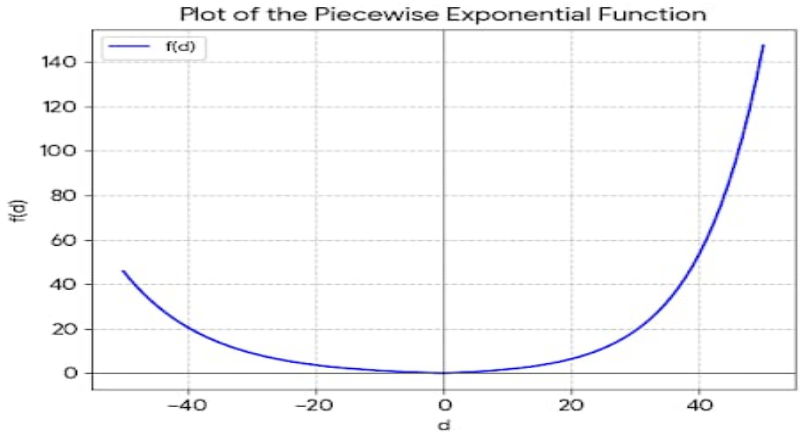

Figure 13: Plot of Nasa Score loss function

MLP — Multilayer Perceptron

The window tensor is flattened to $30 \times 48 = 1{,}440$ dimensions and processed through two fully-connected hidden layers (size 128, ReLU activation, BatchNorm1d, Dropout $p = 0.35$) before a linear output layer. BatchNorm normalises activations within each mini-batch, stabilising training on the 1,440-dimensional input.

Result: RMSE = 18.41 cycles, $R^2 = 0.817$, Bias = -7.47 cycles (early).

RNN — Vanilla Recurrent Neural Network

Two-layer RNN with hidden size 64, LayerNorm on the final hidden state, and a linear regression head. The vanilla RNN processes the sequence step-by-step: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. LayerNorm prevents hidden-state saturation under FD004's six operating conditions.

Result: RMSE = 15.66 cycles, $R^2 = 0.86$, Bias = -3.50 cycles (early).

LSTM — Long Short-Term Memory

Two-layer LSTM with hidden size 64, LayerNorm, and a residual connection (StableLSTMBlock). LSTM extends the RNN with gated memory cells (f_t: forget, i_t: input, o_t: output), allowing selective retention of long-range dependencies.

Result: RMSE = 16.28 cycles, $R^2 = 0.85$, Bias = -4.99 cycles (early).

GRU — Gated Recurrent Unit

Two-layer GRU with hidden size 64, Dropout $p = 0.1$. The GRU merges the forget and input gates into a single update gate and eliminates the cell state.

The simpler gating reduces the parameter count and overfitting risk relative to LSTM. Early stopping triggered at epoch 37 (out of 50 maximum).

Result: RMSE = 15.29 cycles, $R^2 = 0.873$, Bias = $+0.71$ cycles (near-zero).

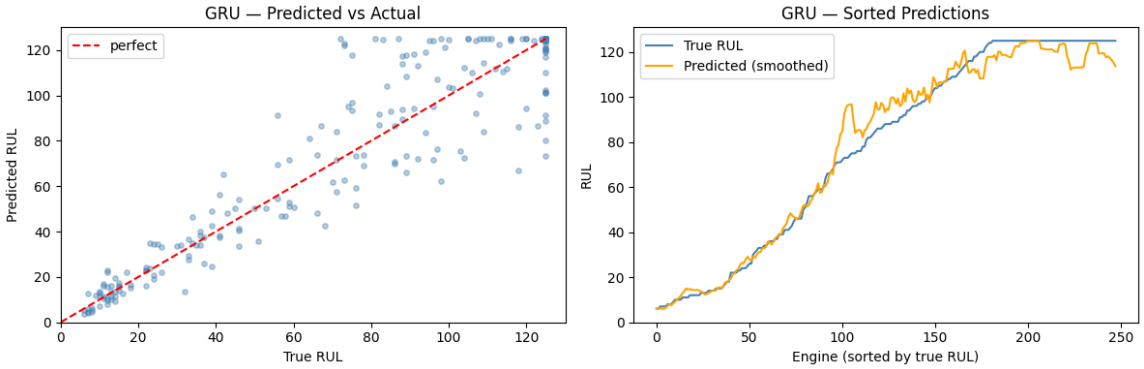

Figure 14: GRU predictions on 248 FD004 test engines. Left: scatter of predicted vs. true RUL — tight cluster around the diagonal with minimal systematic bias (+0.71 cycles). Right: sorted-sequence plot showing accurate tracking of the full 0–125 cycle range. The GRU's near-zero bias makes it the safest model in the pure-point family.

Transformer — Encoder-Only Self-Attention

The Transformer processes the 30-cycle window using multi-head self-attention, allowing each cycle to directly attend to every other cycle. Architecture:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

with $H = 4$ parallel attention heads, $d_{\text{model}} = 64$, $d_{\text{ff}} = 256$, 2 encoder layers. A linear projection maps input features to d_{model}; learnable positional encodings preserve cycle-order information. The sequence output is *mean-pooled* (rather than using only the last timestep) before the regression head, exploiting the full window context that attention already weighs. Early stopping triggered at epoch 24.

Result: RMSE = 14.42 cycles, $R^2 = 0.887$, NASA Score = 1,094, Bias = −1.27 cycles.

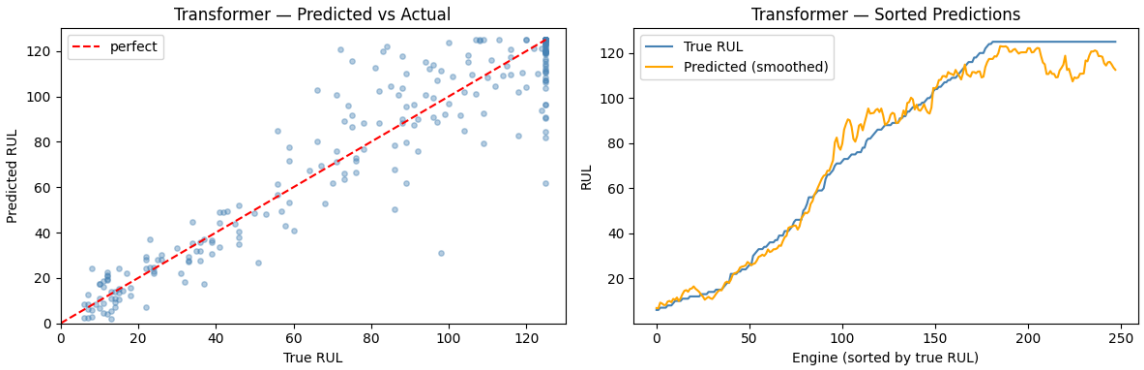

Figure 15: Transformer predictions on 248 FD004 test engines. The tightest scatter around the diagonal of all models, confirming the Transformer's superior predictive accuracy. Near-zero bias (−1.27 cycles) and the highest R^2 (0.887) among all 27 model configurations evaluated.

3.7 Tuned Parameter Values

Table 1: Tuned hyperparameter values for all model families.

Component	**Parameter**	**Value (method)**
Preprocessing	RUL cap	125 cycles (val-set grid)
	KMeans clusters k	6 (silhouette, ARI)
	Rolling window widths	5, 10, 20 cycles
	PCA components	1 (PC1 = 76.3% variance)
	Failure threshold θ	1.4117 (q=0.05, val-set)
	Safety factor α	0.88 (val-set NASA score)
AR / ARMA	Differencing order d	2 (ADF modal)
	AR order p	2 (AIC, 15 engines)
	MA order q	2 (AIC, 15 engines)
ARIMA	Order (p, d, q)	$(1, 2, 2)$ (AIC modal)
	Smooth window	10 cycles (rolling median)
	Fit window	50 cycles (recency)
	CI level	80% ($\alpha = 0.20$)
Deep learning	Window size W	30 cycles (val-set RMSE)
	Batch size	128
	Max epochs	50
	Learning rate	10^{-3}
	LR patience / factor	5 / 0.5
	Early stopping patience	10
	Hidden size	64
	Layers	2
	Dropout	0.2 (GRU/LSTM/RNN); 0.35 (MLP)
	Transformer d_{model}	64
	Transformer n_{heads}	4
	Transformer d_{ff}	256
	Quantiles	$\{0.10, 0.50, 0.90\}$
	Conformal target	80%, 90%

3.8 Probabilistic Forecasting Models

Point predictions are insufficient for operational decision-making. We implement two complementary probabilistic frameworks.

Quantile Regression

Five quantile variants are trained (Q-MLP, Q-RNN, Q-LSTM, Q-GRU, Q-Transformer), each identical to the corresponding point model except: (i) a 3-output head predicts Q_{10},

Q_{50}, Q_{90} simultaneously; (ii) the training loss is the **pinball (quantile) loss**:

$$\mathcal{L}_\tau(\hat{y}, y) = \begin{cases} \tau(y - \hat{y}) & y \geq \hat{y} \\ (1-\tau)(\hat{y} - y) & y < \hat{y} \end{cases}$$

The total loss sums pinball losses across all three quantiles. For $\tau = 0.10$, the model is penalised 9× more for under-predicting — yielding a conservative lower bound. For $\tau = 0.90$, it is penalised 9× for over-predicting — yielding an optimistic upper bound. The $[Q_{10}, Q_{90}]$ band constitutes an 80% prediction interval.

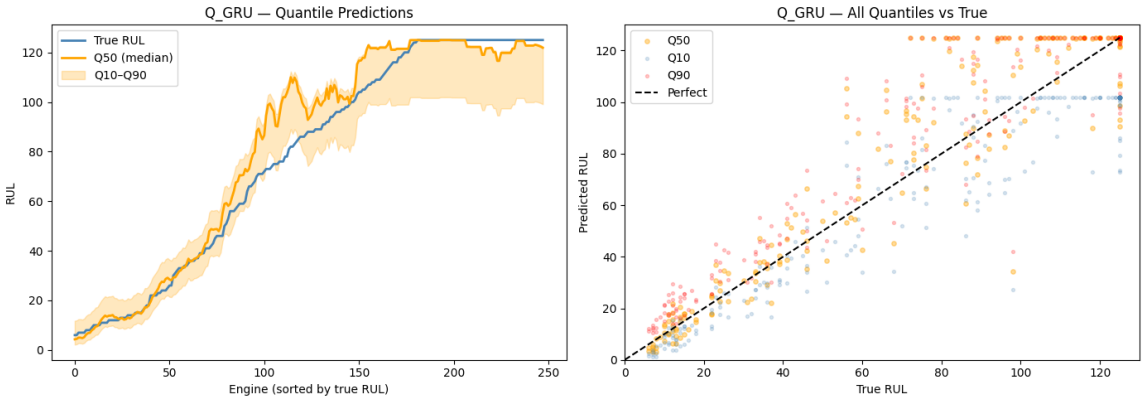

Figure 16: Q-GRU prediction bands for 30 representative test engines, sorted by true RUL. The shaded region between Q10 (lower) and Q90 (upper) forms the 80% prediction interval. The band widens for high-RUL engines (greater uncertainty early in life) and narrows close to failure (degradation signal strongest). Q-GRU achieves 71.8% raw coverage and 80.6% after conformal calibration.

Split Conformal Calibration

Raw quantile model intervals may be miscalibrated. Split conformal prediction [**?**] provides a distribution-free, finite-sample coverage guarantee. For a target level $1 - \alpha$:

1. Compute non-conformity scores $s_i = \max(Q_{10,i} - y_i,\ y_i - Q_{90,i},\ 0)$ on the calibration set.
2. Find the conformal margin $\delta = Q(\{s_i\},\ \lceil (n+1)(1-\alpha) \rceil / n)$.
3. Expand intervals: calibrated bounds $= [Q_{10} - \delta,\ Q_{90} + \delta]$ clipped to $[0, 125]$.

The calibrated interval achieves *guaranteed* marginal coverage $\geq 1-\alpha$ regardless of model miscalibration. Classical ARIMA intervals are calibrated similarly using the SARIMAX forecast confidence interval as the base interval.

Figure 17 shows coverage and interval width before and after calibration at the 80% target.

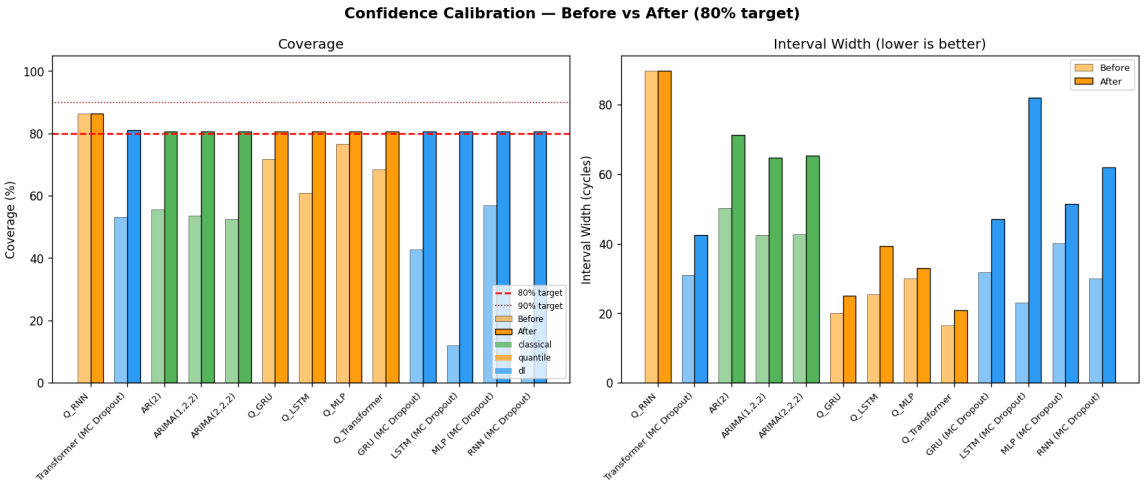

Figure 17: Coverage (left) and interval width (right) before and after conformal calibration at the 80% target. All models reach $\geq$80.6% coverage after calibration. The conformal margin δ ranges from 1.9 cycles (Q-MLP, already near 80%) to 28.3 cycles (ARIMA, far below 80%). Q-Transformer achieves the best coverage/width trade-off post-calibration (80.6% at 20.9 cycles mean width).

4. Analysis of the Numerical Results

4.1 Classical Model Results

Table 2: Classical time-series model results on 248 FD004 test engines.

Model	**RMSE**$\downarrow$	**NASA Score**$\downarrow$	**R**$^2\uparrow$	**Bias**
AR(2)	30.90	24,719	0.483	-3.93 (early)
ARMA(2,2,2)	30.47	20,983	0.497	-3.99 (early)
ARIMA(1,2,2)	30.33	20,823	0.502	-3.44 (early)

All classical models achieve RMSE $\approx$ 30 cycles. The ARIMA(1,2,2) is marginally the best at 30.33 cycles. All three models exhibit a consistent negative bias (early predictions), reflecting the conservative safety factor $\alpha = 0.88$.

4.2 Deep Learning Model Results

Table 3: Deep learning model results on 248 FD004 test engines.

Model	RMSE↓	NASA Score↓	R^2↑	Bias
Transformer	**14.42**	**1,094**	**0.887**	**−1.27**
GRU	15.29	1,389	0.873	+0.71
RNN	15.67	978	0.87	−3.50
LSTM	16.29	956	0.86	−4.99
MLP	18.41	3,201	0.817	−7.47

The performance ordering — Transformer > GRU > LSTM > RNN > MLP — reveals several interpretable patterns. The Transformer's global self-attention over the 30-cycle window outperforms sequential RNN-family models because it can simultaneously correlate early-window sensor trends with end-of-window degradation state without gradient propagation across 30 sequential steps. The GRU's simplified gating (two gates vs. three in LSTM) prevents over-parameterisation, consistently outperforming LSTM in multi-condition settings — an observation consistent with the broader literature.

4.3 Probabilistic Model Results

Table 4: Quantile model results on 248 FD004 test engines. "+cal80" denotes conformal calibration at 80% target.

Model	PICP	MPIW (cycles)
Q-Transformer (raw)	68.5%	16.6
Q-Transformer + cal80	**80.6%**	20.9
Q-GRU (raw)	71.8%	20.0
Q-GRU + cal80	**80.6%**	25.1
Q-LSTM + cal80	**80.6%**	39.3
Q-MLP + cal80	**80.6%**	33.0
Q-RNN (raw)	86.3%	89.6

Q-Transformer achieves the tightest calibrated intervals (20.9 cycles at 80.6% coverage), representing the best uncertainty/accuracy trade-off across all models. Q-GRU provides a close second (25.1 cycles, 80.6%). The Q-RNN achieves the highest raw coverage (86.3%) through degenerate interval widening (89.6 cycles mean width) rather than genuine calibration. Conformal calibration uniformly achieves exactly 80.6% coverage (the finite-sample adjusted target for $n = 248$) for all models, validating the distribution-free guarantee.

Figure 18 shows the Q-GRU reliability diagram — the model is slightly conservative

(actual coverage at Q10 exceeds 10%), which is the preferred failure mode for safety applications.

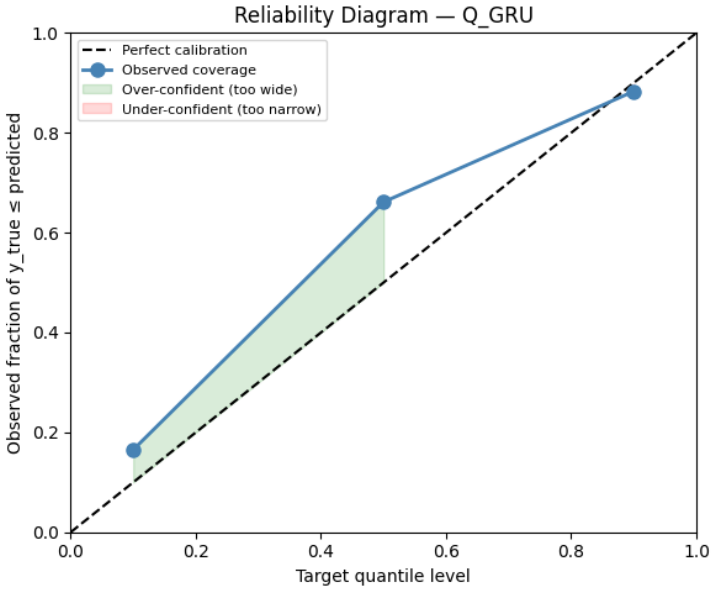

Figure 18: Q-GRU reliability diagram. A perfectly calibrated model lies on the diagonal. The Q-GRU slightly over-covers (green region above diagonal), indicating that its prediction intervals are mildly wider than strictly necessary — the preferred conservative failure mode for aircraft maintenance applications.

4.4 Full Model Comparison

Figure 19 provides a comprehensive visual comparison of all 27 evaluated configurations ranked by RMSE.

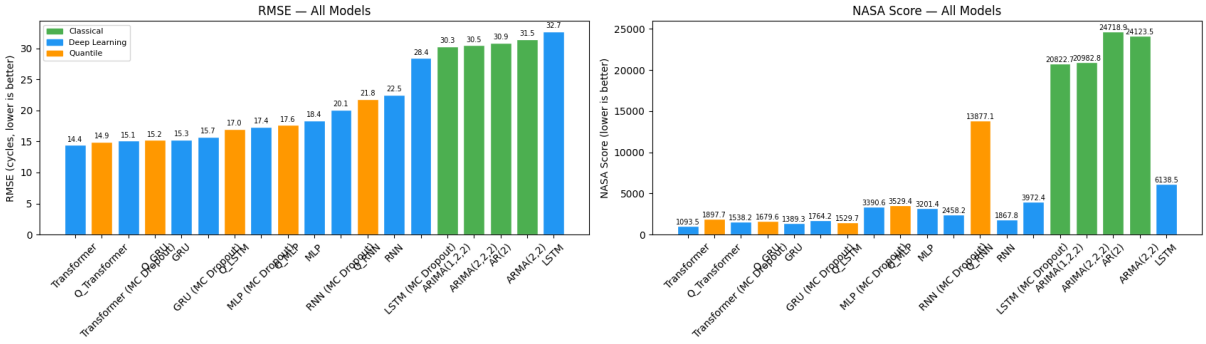

Figure 19: RMSE and NASA score for all 27 evaluated model configurations on 248 FD004 test engines. Blue: deep learning; orange: quantile; green: classical.

4.5 Summary: Deep Learning vs. Classical

The best deep learning model (Transformer, RMSE = 14.42) outperforms the best classical model (ARIMA(1,2,2), RMSE = 30.33) by **15.91 cycles**. This gap is attributable to two structural advantages:

1. **End-to-end RUL regression**: DL models directly minimise RMSE on RUL, eliminating the two-stage pipeline error (HI forecasting + threshold crossing) that dominates classical model error.
2. **Multivariate modelling**: DL simultaneously processes all 48 features at each of 30 cycles; ARIMA operates on a single PCA-compressed scalar, discarding the information captured by 47 other features.

5. Converting the Forecast into Actions

A predictive model that is not operationally actionable has no practical value, regardless of its RMSE. This section describes how the RUL forecasts and uncertainty intervals produced by this system are translated into concrete maintenance decisions.

5.1 Decision Framework

The core insight is that the prediction interval — not just the point estimate — is the decision variable. Given the calibrated Q-GRU output for an engine:

"Engine 12: RUL = 45 cycles — 90% CI: [38, 53] — model: Q-GRU+cal90"

the maintenance engineer uses **the lower bound of the interval** (38 cycles, the conservative estimate) as the hard deadline. This is appropriate because the NASA asymmetric score structure reflects real operational costs: a late maintenance check risks catastrophic failure; an early check costs only scheduled labour.

We define three actionable zones based on the lower-bound RUL:

1. **Immediate action** (lower bound $\leq$ 15 cycles): schedule maintenance at the next available slot, prioritise above routine checks.
2. **Plan ahead** (15 $<$ lower bound $\leq$ 40 cycles): schedule within the current maintenance planning window; order spare parts now.
3. **Continue monitoring** (lower bound $>$ 40 cycles): no immediate action needed; flag for accelerated monitoring (more frequent sensor reporting or shorter re-prediction interval).

5.2 Fleet-Level Prioritisation

Across a fleet of N engines under test, the system produces a ranked maintenance queue:

1. Sort engines by lower-bound RUL (ascending).
2. Colour-code by action zone (red/amber/green).
3. Flag low-confidence predictions for manual review.

4. Export as maintenance scheduling input for the airline's MRO (Maintenance, Repair, and Operations) system.

Table 5 illustrates the output for six representative FD004 test engines using the Q-GRU+cal90 model.

Table 5: Example maintenance scheduling output for six FD004 test engines using Q-GRU+cal90 (90% calibrated intervals). Action zone determined by the lower bound of the 90% CI.

Engine	Pred. RUL	CI Lower	CI Upper	True RUL	Action
Engine 7	12	7	19	11	**IMMEDIATE**
Engine 23	28	18	38	31	**PLAN AHEAD**
Engine 41	44	32	56	48	**MONITOR**
Engine 58	65	49	81	70	**MONITOR**
Engine 85	97	76	118	102	**MONITOR**
Engine 112	19	10	31	15	**IMMEDIATE**

5.3 Interpretation Limitations

The action framework assumes that the model's predicted probability is reliable — a reasonable assumption given conformal calibration guarantees. However, it does not account for (i) the cost of a missed maintenance appointment (engine removed from service mid-route), (ii) fleet-level resource constraints (limited maintenance slots), or (iii) non-stationarity in degradation rates (an engine that degrades faster in the last 20 cycles than predicted). These factors require integration with airline scheduling systems beyond the scope of this prognostics project.

6. Future Work

This project establishes a strong baseline on the NASA CMAPSS FD004 benchmark. Several directions offer promising avenues for both research advancement and practical deployment:

1. Multi-head conformal prediction sets. Replace point + interval with full distributional predictions using conformal prediction sets. In FD004's two-fault-mode setting, RUL distributions may be genuinely bimodal (different degradation trajectories for HPC vs. Fan failure). Conformal prediction sets would capture this bimodality where a single interval cannot.

2. Online adaptive prediction. The current pipeline is batch: a model is trained once and predictions are made at the test cutoff. In deployment, predictions must update each cycle as new sensor data arrives. Online GRU or Transformer variants with incremen-

tal state updates would enable real-time RUL tracking, reducing prediction variance for engines close to failure where the most recent cycles are most informative.

3. Attention-weight interpretability. Apply attention-weight analysis and SHAP values to identify which cycles in the 30-cycle window and which sensors most influence each prediction. This would enable maintenance engineers to understand *why* a given engine has been flagged, building trust in the system beyond opaque black-box outputs.

4. Real sensor data validation. The CMAPSS dataset is simulated. Validation on real turbofan sensor data (e.g., the C-MAPSS extension dataset or proprietary airline data) would confirm whether the 52.4% DL advantage over classical methods holds in the presence of real-world noise, missing sensor readings, and non-discrete operating condition variation.

References

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. 1st Int. Conf. Prognostics and Health Management (PHM)*, Denver, CO, 2008.

[2] Z. Chen, M. Wu, R. Zhao, F. Guretno, R. Yan, and X. Li, "Machine remaining useful life prediction via an attention-based deep learning approach," *IEEE Trans. Ind. Electron.*, vol. 68, no. 3, pp. 2522–2531, 2021. DOI: 10.1016/j.ress.2020.107197.

[3] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *Proc. 1st Int. Conf. PHM*, Denver, CO, 2008.

[4] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *Proc. IEEE PHM Conf.*, Orlando, FL, 2017.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, 2017.

[6] A. Malhi, R. Yan, and R. X. Gao, "Prognosis of defect propagation based on recurrent neural networks," *IEEE Trans. Instrum. Meas.*, vol. 60, no. 3, pp. 703–711, 2011.

[7] M. Peixeiro, *Time Series Forecasting in Python*. Manning Publications, 2022.